Cadmium Experiments

Tick-Counter Reordering Study

Damián Vicino

2026

Contents

1 Implementation Notes

This section records the formalism choices, simulator capabilities, and conventions that apply to every experiment in this repository.

Foundational References

The tick-counter experiment reproduced in this repository originates from [2]. Cadmium [4] is an evolution of the sequential PDEVS architecture [3], adding a compile-time typed port system. PDEVS is defined in [1].

Cadmium PDEVS

Cadmium implements Parallel DEVS (PDEVS) [1] following the sequential call/return architecture of [3] with a compile-time port system introduced in [4]. All simulation steps execute in a single thread with a call stack linear in the model hierarchy depth.

PDEVS transitions *all* imminent models simultaneously. At each time step the set of imminent models $I \subseteq D$ is computed; every model in I fires its output function λ and then its internal transition δ_{int}; models not in I that receive routed output undergo their external transition δ_{ext}. When a model is both imminent and receives external input, the **confluence function** δ_{con} resolves the two transitions:

$$\delta_{\text{con}}(s, e, x) = \delta_{\text{ext}}(\delta_{\text{int}}(s),\ 0,\ x) \quad \text{(internal-first, the Cadmium default)}$$

Multiple simultaneous messages are delivered as a **message bag** (`std::vector<T>` per port), allowing many values per port per step.

Cadmium Classic DEVS

Classic (sequential) DEVS serialises event processing: at each time step *exactly one* imminent model transitions. When more than one model is scheduled at the same time, the **SELECT** function breaks the tie:

$$\text{SELECT} : 2^D \setminus \{\emptyset\} \to D, \quad \text{SELECT}(I) \in I$$

The chosen model fires its output and internal transition; outputs are delivered to influenced models as **message boxes** (at most one value per port per step); then time advances and the next step begins. There is no confluence function in Classic DEVS.

Key Differences

	PDEVS	Classic DEVS
Imminent set	all simultaneous models	one model (SELECT)
Tie-breaking	confluence function	SELECT function
Message carrier	bag (`vector<T>`)	box (at most one)
δ_{con}	defined	not part of the formalism

Port System

Both formalisms in Cadmium use a **strongly typed port system**. Each port is a distinct C++ type carrying a specific message type; couplings are expressed as `(sender-port, receiver-port)` pairs verified at compile time. Port types are declared as:

- `cadmium::out_port<T>` — output port carrying values of type `T`
- `cadmium::in_port<T>` — input port carrying values of type `T`

Each model declares its ports via `using input_ports` and `using output_ports` type aliases. The compiler rejects any coupling where the message types do not match, replacing runtime encoding conventions with compile-time guarantees.

TIME Type

Cadmium is parameterised over a `TIME` template argument that must satisfy `cadmium::concepts::Time`: totally ordered, supporting arithmetic operations, and providing `std::numeric_limits<TIME>::infinity()` for passive time advance.

Common choices:

Type	Representation	Risk
`float`	IEEE 754 single	1/10 not exact; errors may break the causality chain and are unbounded
`double`	IEEE 754 double	same issue, smaller magnitude
rational	exact fraction	no rounding error; heavier compute

Each experiment spec states the TIME types it exercises and the expected impact on simulation correctness.

Log Format

Cadmium emits structured **NDJSON** (Newline-Delimited JSON) to stdout — one JSON object per line. Every record carries a wall-clock timestamp (`ts`),

a severity level (`level`), a structured event name (`event`), and a human-readable message (`msg`). Events during the simulation additionally include `sim_time`, the logical clock value at that instant.

In these experiments the log will contain:

- a `run_info` record at simulation start,
- `sim_state` records per step at `debug` level (state, message routing),
- one record per output emitted by the counter model,
- a `run_info` record at simulation end with wall time.

For the complete event schema and log levels, see `simulators/cadmium/docs/log-format.md`.

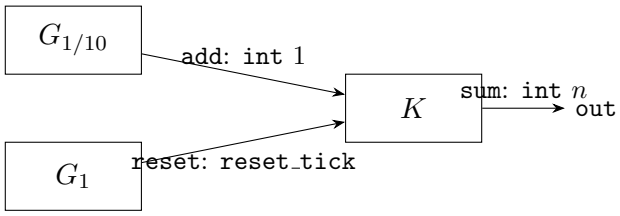
2 VDW14 Tick-Counter — PDEVS

Overview

The tick-counter model simulates a periodic tick source that is counted over time and periodically registered and reset. Two generators produce events at different rates: a fast generator fires every 1/10 sec (the *tick*) and a slow generator fires every 1 sec (the *reset*). A counter model accumulates incoming ticks and, upon receiving a reset signal, outputs the current count and resets to zero. Ideally the counter outputs 10 at every reset.

Under PDEVS, the tick and reset generators are simultaneously imminent at every integer second. Cadmium uses separate typed ports for the two signals, enforcing the routing contract at compile time rather than by value encoding. The experiment reveals whether the chosen TIME type preserves the simultaneity invariant.

Model Diagram



Coupled Model

$$C = \langle X_C,\, Y_C,\, D,\, \{M_d\},\, \text{EIC},\, \text{EOC},\, \text{IC} \rangle$$

$$D = \{G_{1/10},\, G_1,\, K\}, \quad X_C = \emptyset, \quad Y_C \subseteq \mathbb{N}$$

$$\begin{aligned} \text{EIC} &= \emptyset \\ \text{IC} &= \{(G_{1/10}.\texttt{out} \to K.\texttt{add}),\ (G_1.\texttt{out} \to K.\texttt{reset})\} \\ \text{EOC} &= \{(K.\texttt{sum} \to C.\texttt{out})\} \end{aligned}$$

Expected invariant: every output of K equals 10.

Atomic Model Definitions

Generator $G_{1/10}$.

$$G_{1/10} = \langle X,\, Y,\, S,\, \delta_{\text{ext}},\, \delta_{\text{int}},\, \delta_{\text{con}},\, \lambda,\, ta \rangle$$

$$
\begin{aligned}
X &= \emptyset \\
Y_{\texttt{out}} &= \{1\} \\
S &= \left[0,\ \tfrac{1}{10}\right) \\
\delta_{\text{ext}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\delta_{\text{int}}(s) &= 0 \\
\delta_{\text{con}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\lambda(s) &= (\texttt{out} : 1) \\
ta(s) &= \left(\tfrac{1}{10} - s\right) \text{sec}
\end{aligned}
$$

Implementation. `basic_model::pdevs::generator<TIME>` from namespace `cadmium`; see `tick_gen.hpp`. S represents elapsed time; since $X = \emptyset$, $s = 0$ always holds at transition time and ta returns the constant $\frac{1}{10}$ sec. Port `out`: `out_port<int>`; value transported is 1 (tick). No EIC connection — $X = \emptyset$.

Generator G_1.

$$G_1 = \langle X,\ Y,\ S,\ \delta_{\text{ext}},\ \delta_{\text{int}},\ \delta_{\text{con}},\ \lambda,\ ta \rangle$$

$$
\begin{aligned}
X &= \emptyset \\
Y_{\texttt{out}} &= \{\texttt{reset\_tick}\} \\
S &= \left[0,\ 1\right) \\
\delta_{\text{ext}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\delta_{\text{int}}(s) &= 0 \\
\delta_{\text{con}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\lambda(s) &= (\texttt{out} : \texttt{reset\_tick}) \\
ta(s) &= \left(1 - s\right) \text{sec}
\end{aligned}
$$

Implementation. `basic_model::pdevs::generator<TIME>` from namespace `cadmium`; see `reset_gen.hpp`. S represents elapsed time; ta returns the constant 1 sec. Port `out`: `out_port<reset_tick>`; value is a `reset_tick` sentinel (unit type with no payload — the signal is the arrival itself). No EIC connection — $X = \emptyset$.

Counter K.

$$K = \langle X,\ Y,\ S,\ \delta_{\text{ext}},\ \delta_{\text{int}},\ \delta_{\text{con}},\ \lambda,\ ta \rangle$$

$$
\begin{aligned}
X_{\texttt{add}} &= \mathbb{N} \qquad\qquad & X_{\texttt{reset}} &= \{\texttt{reset\_tick}\} \\
Y_{\texttt{sum}} &= \mathbb{N} \\
S &= \mathbb{N} \times \{\text{idle},\ \text{ready}\}
\end{aligned}
$$

Let $x_{\texttt{add}}$ denote the PDEVS bag on port add, $x_{\texttt{reset}}$ the bag on port reset.

$$ta(\bullet, \text{idle}) = +\infty\,\text{sec} \qquad ta(\bullet, \text{ready}) = 0\,\text{sec}$$

$$\delta_{\text{ext}}((n, \text{mode}), e, x) = \begin{cases} (n + |x_{\texttt{add}}|,\ \text{ready}) & x_{\texttt{reset}} \neq \emptyset \\ (n + |x_{\texttt{add}}|,\ \text{idle}) & x_{\texttt{reset}} = \emptyset \end{cases}$$

$$\delta_{\text{int}}(n, \text{ready}) = (0,\ \text{idle})$$
$$\delta_{\text{con}}(s, e, x) = \delta_{\text{ext}}(\delta_{\text{int}}(s),\ 0,\ x)$$
$$\lambda(n, \text{ready}) = (\texttt{sum} : n)$$

Implementation. `basic_model::pdevs::accumulator<int,TIME>` from namespace `cadmium`. S is stored as `(n: int, mode)` with an internal enum {`idle, ready`}. Port `add`: `in_port<int>`; each arriving `int` increments n. Port `reset`: `in_port<reset_tick>`; arrival triggers the ready state regardless of payload. Port `sum`: `out_port<int>`; emits n on internal transition.

Simulation Setup

Both TIME variants are driven by the Cadmium `pdevs::runner` with a fixed target time limit. The runner advances the simulation until logical time reaches the limit. If the two runs produce logs of different length (e.g. due to causality breaks in the float run), the longer log is truncated to the shorter run for comparison.

TIME types exercised. `float` and a rational type.

Expected Observations

Float time: IEEE 754 cannot represent 1/10 exactly. Floating-point error shifts occasional $G_{1/10}$ ticks across a 1-second boundary; K outputs 9 or 11. Errors may break the causality chain and are unbounded. Observed: ≈11.4% error rate over 100 000 resets (histogram: $9 \times 10\,406$, $10 \times 88\,624$, 11×970).

Rational time: $G_{1/10}$ and G_1 are always simultaneously imminent at integer seconds; confluence correctly resolves the tie and K always outputs 10.

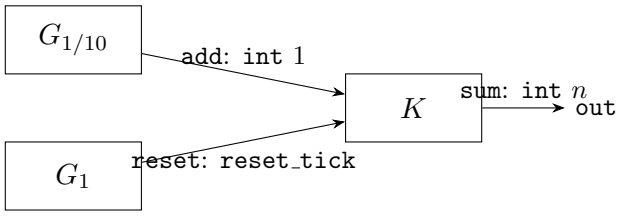
3 VDW14 Tick-Counter — Classic DEVS

Overview

The model is the same tick-counter as in the PDEVS section: a fast generator fires every 1/10 sec (tick), a slow generator fires every 1 sec (reset), and a counter accumulates ticks and outputs the count on reset. Ideally the counter outputs 10 at every reset.

Under Classic DEVS, at most one model transitions per step. When the tick and reset generators are simultaneously imminent (at integer seconds under exact arithmetic), the **SELECT** function must order them to guarantee the correct count. The experiment reveals how the interaction between SELECT ordering and TIME type affects the invariant.

Model Diagram



Coupled Model

$$C = \langle X_C,\, Y_C,\, D,\, \{M_d\},\, \text{EIC},\, \text{EOC},\, \text{IC},\, \text{SELECT}\rangle$$

$$D = \{G_{1/10},\, G_1,\, K\}, \quad X_C = \emptyset, \quad Y_C \subseteq \mathbb{N}$$

$$\begin{aligned} \text{EIC} &= \emptyset \\ \text{IC} &= \{(G_{1/10}.\texttt{out} \to K.\texttt{add}),\ (G_1.\texttt{out} \to K.\texttt{reset})\} \\ \text{EOC} &= \{(K.\texttt{sum} \to C.\texttt{out})\} \end{aligned}$$

$$\text{SELECT}(I) = \begin{cases} G_{1/10} & G_{1/10} \in I \\ G_1 & G_1 \in I,\ G_{1/10} \notin I \\ K & \text{otherwise} \end{cases}$$

SELECT must prioritise $G_{1/10}$ so that K accumulates the final tick before receiving the reset. Expected invariant: every output of K equals 10.

Atomic Model Definitions

Classic DEVS atomic models have no confluence function; the tuple is $\langle X, Y, S, \delta_{\text{ext}}, \delta_{\text{int}}, \lambda, ta\rangle$.

Generator $G_{1/10}$.

$$G_{1/10} = \langle X,\ Y,\ S,\ \delta_{\text{ext}},\ \delta_{\text{int}},\ \lambda,\ ta \rangle$$

$$\begin{aligned}
X &= \emptyset \\
Y_{\texttt{out}} &= \{1\} \\
S &= \left[0,\ \tfrac{1}{10}\right) \\
\delta_{\text{ext}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\delta_{\text{int}}(s) &= 0 \\
\lambda(s) &= (\texttt{out} : 1) \\
ta(s) &= \left(\tfrac{1}{10} - s\right) \text{sec}
\end{aligned}$$

Implementation. `basic_model::devs::generator<TIME>` from namespace `cadmium`; see `tick_gen.hpp`. S represents elapsed time; ta returns the constant $\frac{1}{10}$ sec. Port `out`: `out_port<int>`; value transported is 1 (tick). No EIC connection — $X = \emptyset$.

Generator G_1.

$$G_1 = \langle X,\ Y,\ S,\ \delta_{\text{ext}},\ \delta_{\text{int}},\ \lambda,\ ta \rangle$$

$$\begin{aligned}
X &= \emptyset \\
Y_{\texttt{out}} &= \{\texttt{reset_tick}\} \\
S &= \left[0,\ 1\right) \\
\delta_{\text{ext}} &= \bot \quad \text{(undefined; } X = \emptyset\text{)} \\
\delta_{\text{int}}(s) &= 0 \\
\lambda(s) &= (\texttt{out} : \texttt{reset_tick}) \\
ta(s) &= \left(1 - s\right) \text{sec}
\end{aligned}$$

Implementation. `basic_model::devs::generator<TIME>` from namespace `cadmium`; see `reset_gen.hpp`. S represents elapsed time; ta returns the constant 1 sec. Port `out`: `out_port<reset_tick>`; value is a `reset_tick` sentinel (unit type — the signal is the arrival). No EIC connection — $X = \emptyset$.

Counter K.

$$K = \langle X,\ Y,\ S,\ \delta_{\text{ext}},\ \delta_{\text{int}},\ \lambda,\ ta \rangle$$

$$\begin{aligned}
X_{\texttt{add}} &= \mathbb{N} & \qquad X_{\texttt{reset}} &= \{\texttt{reset_tick}\} \\
Y_{\texttt{sum}} &= \mathbb{N} \\
S &= \mathbb{N} \times \{\text{idle},\ \text{ready}\}
\end{aligned}$$

In Classic DEVS, message boxes deliver at most one value per port per step; let $x_{\texttt{add}} \in \mathbb{N} \cup \{\bot\}$ and $x_{\texttt{reset}} \in \{\texttt{reset\_tick}, \bot\}$.

$$ta(\bullet, \text{idle}) = +\infty \,\text{sec} \qquad ta(\bullet, \text{ready}) = 0 \,\text{sec}$$

$$\delta_{\text{ext}}((n, \text{idle}), e, x) = \begin{cases} (n+1,\ \text{idle}) & x_{\texttt{add}} \neq \bot \\ (n,\ \text{ready}) & x_{\texttt{reset}} \neq \bot \end{cases}$$

$$\delta_{\text{int}}(n, \text{ready}) = (0,\ \text{idle})$$

$$\lambda(n, \text{ready}) = (\texttt{sum} : n)$$

Implementation. `basic_model::devs::accumulator<int,TIME>` from namespace `cadmium` using `make_message_box` (optional-per-port, not a bag). S is stored as `(n: int, mode)` with an internal enum {`idle, ready`}. Port `add`: `in_port<int>`; one arriving `int` increments n by 1 (in this experiment the value is always 1). Port `reset`: `in_port<reset_tick>`; arrival triggers the ready state. Port `sum`: `out_port<int>`; emits n on internal transition.

Processing Order at Integer Seconds

Resulting event sequence at each integer second $t = k$ under SELECT:

1. $G_{1/10}$ fires: routes 1 to K.`add`; K applies δ_{ext} — accumulator reaches 10.
2. G_1 fires: routes `reset_tick` to K.`reset`; K applies δ_{ext} — transitions to ready, $ta_K = 0$.
3. K fires internally: λ_K outputs 10, accumulator resets to 0.

Simulation Setup

Both TIME variants are driven by the Cadmium Classic DEVS runner with a fixed target time limit. If the two runs produce logs of different length, the longer log is truncated to the shorter run for comparison.

TIME types exercised. `float` and a rational type.

Expected Observations

Float time: floating-point error makes $G_{1/10}$ and G_1 non-simultaneous; SELECT is not invoked because only one model is imminent. If G_1 fires before $G_{1/10}$ due to accumulated error, K outputs 9; if $G_{1/10}$ fires one step late, K outputs 11. Errors may break the causality chain and are unbounded.

Rational time: $G_{1/10}$ and G_1 are always simultaneously imminent at integer seconds; SELECT correctly resolves the tie and K always outputs 10.

Key contrast with PDEVS: SELECT only has observable effect when events are *exactly* simultaneous. Float arithmetic makes near-simultaneous events non-simultaneous, rendering SELECT irrelevant for the error case — the bug arises before SELECT is consulted.

References

[1] Bernard P. Zeigler, Herbert Praehofer, Tag Gon Kim. *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems.* Academic Press, 2000.

[2] Damián Vicino, Olivier Dalle, Gabriel A. Wainer. *A Data Type for Discretized Time Representation in DEVS.* In *Proc. 7th Intl. ICST Conf. on Simulation Tools and Techniques (SIMUTOOLS)*, 2014. `hal-01055555`.

[3] Damián Vicino, Daniella Niyonkuru, Gabriel A. Wainer, Olivier Dalle. *Sequential PDEVS Architecture.* In *Proc. Spring Simulation Conference (SpringSim)*, 2015.

[4] Laouen Belloli, Damián Vicino, Cristina Ruiz-Martín, Gabriel A. Wainer. *Building DEVS Models with the Cadmium Tool.* In *Proc. Winter Simulation Conference (WSC)*, 2019.